

HOW TO READ AND JUDGE OPEN-SOURCE ML-KEM: WHAT MAKES POST-QUANTUM CRYPTO SO HARD TO SHIP

BY **Melissa Kilby**

Nero Duality · v2026.09.10

SLIDE 1

There's a lot of talk about post-quantum crypto; today we spend some time reading the code. We open an open-source Post-Quantum Cryptography (PQC) library and learn how to read and judge it: the C, mapped to the standard it implements.

SLIDE 2

I'm Melissa, Founder and security engineer. Welcome to the Hacker Lounge.

SLIDE 3

We use mlkem-native 2.0.0. The name matters: Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM) doesn't encrypt your data; it establishes a 32-byte shared secret that a protocol like Transport Layer Security (TLS) inserts into its key schedule.

SLIDE 4

mlkem-native is the Post-Quantum Cryptography Alliance's ML-KEM library, supported by the Linux Foundation: a hardened fork of the Kyber/ML-KEM reference code.

SLIDE 5

Next slide is just orientation: several places ML-KEM shows up; some import mlkem-native, some ship their own; all target the Federal Information Processing Standard (FIPS) 203, ...

SLIDE 6

... then we stay on the mlkem-native library.

SLIDE 6.1

This talk is for anyone curious about post-quantum crypto: reading an ML-KEM library, seeing how the C maps to the spec, or learning what to watch out for when you integrate, whether that's TLS, firmware, or anywhere else the KEM shows up.

SLIDE 6.2

This is about a one-hour deep-dive video. We take the title in reverse: the what-makes-it-so-hard-to-ship half first, then the code reading in Part 2, and that second part gets dense. Recommended to pause between slides or chapters and come back. Want to jump straight to the code? Skip ahead about 12 minutes.

SLIDE 7

Up next: what brings us to post-quantum, the new standards, and how we ship it.

SLIDE 8

We did all this work. We encrypted the entire internet. Out of all the hard math problems in the universe, we had to pick the two that quantum computers use as cheat codes.

SLIDE 9

Quantum computers aren't general-purpose speedups. They're specialized machines, and a cryptographically relevant one (meaning a large, error-corrected machine capable of running attacks like Shor's) doesn't exist yet. Timelines keep moving.

When one does: classical public-key crypto for key exchange and signatures – gone.

For long-lived secrets there's a nastier twist (harvest now, decrypt later): an attacker records your encrypted traffic today and simply waits for a future machine to open it.

SLIDE 10

No comment.

SLIDE 11

1976: Diffie-Hellman key exchange. This was a big deal. Two strangers can now establish a secret while the whole world is listening.

1977: Rivest-Shamir-Adleman (RSA). Diffie-Hellman gave you key agreement; RSA lets you publish a public key so anyone can encrypt to you, and only you can decrypt.

1985: Elliptic-curve crypto (Koblitz and Miller). Not a new key-agreement protocol: Diffie-Hellman on a different group. Same pattern, much smaller keys. Elliptic Curve Diffie-Hellman (ECDH) becomes the workhorse of pre-quantum key agreement.

1994: Shor's algorithm. A quantum computer that can run it breaks both hardnesses we've been relying on: factoring for RSA, and discrete log for Diffie-Hellman, whether classical or elliptic-curve.

SLIDE 12

As the story goes, Shor found the discrete-log break first and factoring soon after.

One algorithm, both problems. The ones we built the encrypted internet on.

SLIDE 13

Spooky timing: Netscape was about to encrypt the web. The formula that can break that future was already public.

SLIDE 14

February 1995: Netscape ships Secure Sockets Layer (SSL) 2.0, encryption for the web. By 1999, the Internet Engineering Task Force turns that lineage into an open standard: TLS 1.0.

SLIDE 15

Fast-forward: standards catch up.

SLIDE 16

2016: the National Institute of Standards and Technology (NIST) opens the post-quantum competition.

2022: first primary quantum-resistant picks. Same year: KEM backups move into a fourth round, and a call for more signatures to diversify the portfolio.

2024: the big three land as standards: FIPS 203 ML-KEM, 204 Module-Lattice-Based Digital Signature Algorithm (ML-DSA), 205 Stateless Hash-Based Digital Signature Algorithm (SLH-DSA).

None of them lean on factoring or discrete log. ML-KEM's hardness is a lattice problem, Module Learning with Errors (MLWE), and no known quantum algorithm cracks it.

SLIDE 17

Different math, same urgency: keep the caution in the standards room, not on the wire.

NIST's line is blunt: no need to wait; these can and should be put into use now, while evaluation of backup algorithms continues.

SLIDE 18-19

Understood. Encrypt the internet again, better and right now.

Here's how the big deployments ship it.

We don't replace classical key exchange overnight; we go hybrid: combine the old and the new in one handshake, because each defends against a different failure.

Cloudflare, and most of the majors, keep X25519 (ECDH that TLS already uses) and pair it with ML-KEM, usually ML-KEM-768. If ML-KEM ever has a flaw, the classical leg still holds; lose that too, and the flaw is the nail in the coffin.

This slide is about scale, not equal security.

RSA and elliptic-curve stay compact: classical strength.

ML-KEM is the jump: much bigger encapsulation (public) keys and ciphertexts; for ML-KEM-768, 1184 and 1088; shared secret still 32 bytes, ordinary symmetric key material. By the way, symmetric is the quiet survivor here: Grover's algorithm (quantum brute-force search) only square-roots the search, on paper cutting a key's bit-strength in half. So you move up to a bigger key (Advanced Encryption Standard AES-256, to keep a 128-bit security margin), not to a whole new algorithm for the bulk encryption that follows the asymmetric key exchange.

Lastly, for hybrid (ML-KEM and elliptic-curve together), you pay more bytes on the wire. The cost is mainly size and CPU, not a new kind of session key. So that's settled then. NIST says go, the shared secret is still 32 friendly bytes, it's only a kilobyte more on the wire. Ship it Friday.

... Yeah, about that.

What makes post-quantum crypto SO hard to ship? Five stacks. The table's on screen; here's what bites in each.

New algorithms, real uncertainty. Newer than RSA, backups still landing. Historically these migrations take a decade or two. Modern tooling and automation may shorten that, but it's a program, not a library swap.

Engineering surface. Encapsulation (public) keys and ciphertexts jump from 32 bytes to about a kilobyte each, so every buffer, record size, and key store built for tiny keys has to grow, and the hybrid handshake runs two key exchanges.

And a bare ML-KEM library like this one hands back work that a full crypto library would normally bundle for you. `mlkem-native` ships only a `randombytes` prototype, a declaration with no body, so you supply the cryptographically secure pseudo-random number generator (CSPRNG) it calls. Same class of hook a bare X25519 library left you; OpenSSL used to hide it.

FIPS also requires every intermediate value to be wiped. The library ships a default wipe (a `memset` behind a compiler barrier), but on a toolchain without inline assembly, or with assembly switched off, the build stops and that secure-wipe is your code to write.

Wire and middleboxes. A hybrid ClientHello often no longer fits in one packet, and legacy inspection gear silently drops it. Chrome and Cloudflare hit exactly this when they switched hybrid on: Chrome offered a temporary enterprise opt-out while middleboxes got patched; Cloudflare staged the rollout and let customers opt out by zone.

Constrained devices. Firmware, secure elements, and hardware security modules (HSMs) were built for 32-byte elliptic-curve keys and small stacks, and ML-KEM asks for more than they were sized to give.

Decapsulation can peak past 12 kilobytes of stack: `pqm4`'s clean C on Cortex-M4 measures ML-KEM-768 decaps at about 14 KB (stack-tuned asm is far smaller), and `mlkem-native`'s portable C build measures about 16 on a 64-bit host, with a heap-allocation hook when the stack budget is tight. And without hardware-accelerated Secure Hash Algorithm 3 (SHA-3), KECCAK hashing dominates the cycle count. Public-Key Cryptography Standards number eleven (PKCS#11) and more recent hardware security module firmware are starting to expose ML-KEM, but many fielded modules and legacy application programming interfaces (APIs) still don't.

And the quiet one: broken versus bypassed. The math can be perfect and you still lose. Crypto usually isn't broken. It's bypassed.

So let's make "bypassed" concrete. A correct, FIPS-conformant ML-KEM can still lose to everything around it (key management, memory, protocol, the random number generator (RNG), side channels), and since no one can enumerate every attack, treat these as examples. They bite hardest on static or reused keys.

Simple decapsulation key theft. Steal a decapsulation (private) key and you can recover every shared secret under it; for a static key, every recorded ciphertext past and future alike. And in a software implementation, while the code runs it isn't vaulted anywhere: plain bytes in memory. Wipe when done, or a core dump leaks them.

Private seed theft. Or go for the private seeds, if someone kept them. Allowed under FIPS, and those two seeds are enough to rebuild the whole keypair. Guard them like the decapsulation key. Weak randomness is the same failure from the other side: if the seeds were predictable, the attacker never has to steal anything.

Steal the plaintext. Transport encryption only protects data while it is in flight. Once the endpoint decrypts it, that data sits as plaintext in the application's memory. So the practical attacker ignores the handshake and goes for the result on the machine: scrape memory, hook the crypto calls, lift session tokens. Nothing post-quantum about this one; it is the classic endpoint bypass, and ML-KEM does nothing to change it.

Side channels. This is the subtlest class, and it is why much of the code looks the way it does. Steal nothing from memory; just measure timing, power, or cache behavior, sometimes over many queries and sometimes from a single trace, and you can still recover the secret. The defense against the timing and cache channels is constant-time code: code whose branches and memory addresses never depend on the secret. Power and electromagnetic leakage need further countermeasures on top, out of scope today.

FIPS conformance is not the same as constant-time security. That is a separate discipline, and Part 2 shows how the mlkem-native code enforces it.

That's bypass: the math may hold, the handling may not.

SLIDE 30

That answers what makes post-quantum crypto SO hard to ship: five stacks.

Now: how to read and judge open-source ML-KEM.

SLIDE 31

We open the code,

SLIDE 32

FIPS 203, and other important specs.

SLIDE 33

As we open them, keep three questions in mind.

One: how do you judge an ML-KEM library, not just the algorithm, but the configs, glue, and tests around it?

Two: where does the C go beyond the pseudocode, and which of those are load-bearing for security, for performance, or both?

Three: how does ML-KEM actually run in a handshake: Alice and Bob through a simple key establishment, pointed at the real functions?

SLIDE 34

The shippable unit is the mlkem/ folder: headers, about a dozen C files under src/, optional native/ assembly. Everything else is upstream's, not your product tree, and that clean boundary is itself a quality signal. Each file has one job.

SLIDE 35

Same tree, now with one job tag per file. The point isn't the individual rows; it's that they cluster: a little build glue, and everything else falling cleanly onto the FIPS 203 layers. That clustering is the real map.

SLIDE 36

Here's that map. We'll walk it the way FIPS lays it out, inside out: auxiliaries and the Kyber public-key encryption core (K-PKE) first, the external API (KeyGen, Encaps, Decaps) last. The bottom bar is the ship glue that turns the layers into a binary.

What do you open first: the algorithms, or the parameters? Parameters. Section 8: the numbers under every layer.

On the card: `PARAMETER_SET` picks the level (512, 768, or 1024) and sets the module rank `MLKEM_K`. `k` is the lever.

The other level knobs follow: `eta1` (the noise width for the secret and its noise at KeyGen, and for the ephemeral secret at Encaps) plus the compression widths `du` and `dv`. The card also pins `eta2`, the noise width for the encryption errors, at 2 for every level.

And two constants everything else hangs off never move: `n` and `q`. `n` never changes; `k` does: that is how you select NIST security category 1, 3, or 5 (ML-KEM-512, 768, 1024).

Okay, but what even are those?

`n` in the code: one line that matters. An array of exactly 256 integers in a row: that array is one polynomial. The wrap-around rule (`X` to the 256 equals minus 1: shift past the last slot and you wrap back to the start with a sign flip) is what keeps every product inside the same 256 slots: multiply two of these and you get another one, not one nearly twice as long. That closure is exactly what the Number-Theoretic Transform (NTT) exploits – we will unpack that later.

So `n` is the length of one polynomial: 256 slots, each an integer coefficient.

What sets `n` to 256? The key you want out is 256 bits, and each slot carries one bit of the 32-byte message `m` that the key is derived from. So `n` is set by the output; everything else scales around it.

That answers the length, not the cast: most polynomials in ML-KEM are not message carriers. Same 256-slot shapes everywhere, but those slots mean different things depending on their jobs, and it pays to keep those meanings apart. Underneath, it is a game of many polynomials for one purpose: hiding a secret. The code and the spec give that secret the most original name: the letter `s`.

Intuition only for now: which polynomials show up in KeyGen, Encaps, and Decaps, and what the integers in those slots mean in each case. Exact handshake math comes later. Number examples below are for ML-KEM-768 ($k = 3$).

KeyGen: same 256-slot shapes, these jobs:

One: the secret s : a vector of k polynomials (3×256).

Two: noise e on that secret: another vector of k polynomials (another 3×256).

Three: a big public coefficient matrix A : a $k \times k$ grid of random-looking polynomials, so that there is a public noisy linear system to hang the secret on; no matrix A , no hiding (so $3 \times 3 \times 256$ it is).

Four: the public vector t : also k polynomials, t equals A times s plus e (3×256 , not another 3×3).

So KeyGen works with 18 polynomials, 4,608 coefficients in total.

Encaps: same 256-slot shapes, different jobs:

One: the message m : one polynomial (1×256 at every level); each slot still holds one integer, but encodes only one bit of the 32-byte message.

Two: the ciphertext: u is a vector of k polynomials, v is one more ($3 \times 256 + 256$); both are built from the public key, fresh noise, and that message polynomial.

So Encaps works with 24 polynomials, 6,144 coefficients in total: those five just mentioned, plus everything rebuilt or sampled to get there.

The shared secret K is also 32 bytes, but it never sits in a polynomial slot; both sides hash it out of that message.

Decaps: same 256-slot shapes again. 32 polynomials, 8,192 coefficients; details later; intuition only for now.

Back to the Encaps message m : what rules out another value for n ? One bit per slot: a 0 encoded as the integer 0, a 1 as roughly q over 2 (1665) puts those two values as far apart as they can get, so it tolerates the most noise. A smaller n would have to pack several bits into one slot, halving that spacing and forcing weaker noise: less security. And once n is frozen, security scales by stacking k of these polynomials: dimension k times 256 is 512, 768, or 1024. A larger n would make those steps coarser and the smallest set bigger than it needs to be. So: do not shrink n , stack k .

q : the modulus on every coefficient. What sets q to 3329? You want q as small as possible, for two reasons. First, bandwidth: when the code packs the encapsulation key, every coefficient of the public vector t is a fixed-width field – 12 bits, because q is 3329. A larger q needs a wider field, so public keys get bigger. Second, security grows with how big the noise is relative to q . The catch: the scheme is noisy by design, and messages are recovered by rounding away small noise. If the noise ever grows past a quarter of q , a bit flips and honest decryption fails. So q also has to be big enough that this almost never happens.

Add the transform's constraint. NTT-friendly here means 256 divides q minus 1, which leaves the primes 257, 769, 3329, 7681, and up. At 257 and 769 there is too little room for the noise: you cannot get a negligible honest-failure rate, which the CCA security argument needs. Round-one Kyber used 7681; round two dropped to 3329 once an incomplete NTT was shown to be just as fast, to save bandwidth. 3329 is the smallest survivor: at ML-KEM-768 the failure rate is about 2 to the minus 164.8 . That it fits in 12 bits is a bonus the code leans on; the card next shows Barrett, SampleNTT, and ByteEncode_12 as they appear in the code.

q in the code: one constant, all over the library. Three jobs on this card.

First: Barrett reduction. Same job as a remainder (modulo q), without the ordinary modulo $\%$ operator: multiply by a constant, shift, subtract. No divide, so no data-dependent cycle count. The loop under it runs that on every coefficient, then folds each result back into residues 0 to q minus 1.

Second: SampleNTT, the accept gate. It pulls 12-bit chunks from Secure Hash Algorithm KECCAK (SHAKE), an extendable-output function (XOF); we'll unpack SHAKE later. It keeps a chunk only when it is less than q , and otherwise rejects it. Just reducing modulo q would bias the low values; rejecting gives exactly uniform coefficients, and the Module Learning with Errors (MLWE) hardness assumption is that A is uniform. A biased A is a different, unanalyzed problem. Only about 81 percent of chunks pass (3329 of 4096), so accepts can run short and you squeeze more output. That is the reason expanding A needs an XOF.

Third: ByteEncode_12. Same 12-bit width, packing job: every coefficient becomes 12 bits on the wire, and the byte size on the card is just that product.

Two constants that never move, and a lot hanging off them. Take a breath.

SLIDE 44

Next: the knobs that do move when you ship. One config header steers the deployment story: performance, portability, what you expose, what you hook. We'll move fast. For each group, just the knobs that change a shipping decision.

SLIDE 45

API surface: what you expose. `PARAMETER_SET` picks the level, and a namespace prefix keeps two bundled copies from colliding. Headline: shrink the build to the operation you ship. Decaps-only drops keygen and encaps, so code size and attack surface are reduced. The default keeps the full randomized surface; strip that only on purpose.

SLIDE 46

API extras: three that matter. A FIPS 140-3 keygen self-test for validated modules (one encaps-decaps round trip that must agree); a single-file build where the library's symbols can stay private so they do not collide with yours; and an opaque context pointer on every call for your hooks. Plus one tell that isn't a macro: the portable code still targets the 1990 ISO C language standard (C90), a deliberate portability choice, and a sign of how many embedded toolchains still speak nothing newer.

SLIDE 47

Multi-level. One binary can speak all three security levels, sharing common code so you don't ship three copies of the helpers. Firmware and multi-tenant crypto-module territory; a TLS stack just picks one level and moves on.

SLIDE 48

Speed and backends: mix-and-match for performance. Swap portable C for hand-written assembly (Advanced Vector Extensions 2 (AVX2), Arm NEON) on hot pieces like the Number-Theoretic Transform (NTT) and FIPS 202 hashing (SHA-3, SHAKE). Carry this with you: security and side-channel proofs attach to the backend you ship, not to the C code you read.

SLIDE 49

Platform hooks, and look how many. Cryptographic RNG, secure-wipe, allocator, even `memcpy` and `memset`: config options let you replace each with your own code. Two are non-negotiable in production: a real RNG and a working zeroize; the rest are optional when the defaults do not fit.

Test and compliance. One flag turns on automated constant-time checking under Valgrind; another deliberately breaks the keygen self-test so you can prove failure handling fires. That's the config tour.

Now the top of the map lights up: the four algorithm layers: the external ML-KEM, the internal algorithms, the Kyber public-key encryption core (K-PKE), and the auxiliaries beneath. FIPS lays them out inside out, and so will we, which means, before Alice and Bob and the actual API calls, ...

... there's a bit more confusing math to get out of the way: the auxiliaries. Hashes, sampling, compression, the NTT. Dry on their own, but each one carries a real security or performance consequence, so it's worth the quick detour.

The auxiliary algorithms underpin everything else, and they're really an extension of that engineering surface we opened with. Get familiar with these first and the KeyGen, Encaps, and Decaps walk at the end falls out in no time.

ByteEncode, ByteDecode, Compress, Decompress, in compress.c. These are the bridges between polynomials in memory and bytes on the wire, and they come in two flavors.

ByteEncode and ByteDecode are lossless, pure bit packing at a width d ; ByteEncode₁₂ is the one we met at MLKEM_Q: since q is under 2^{12} , every coefficient fits in 12 bits, so one polynomial packs to 256 times 12 bits, 384 bytes (MLKEM_POLYBYTES), and unpacks bit-for-bit. Keys use it: the public vector t and the secret vector s have to come back exact.

Compress and Decompress are the lossy pair, run on the ciphertext before packing: Compress rescales each coefficient from modulo q down to d bits, keeping roughly the top du bits of each coefficient of u (10 or 11) and the top dv bits of each coefficient of v (4 or 5), rounding the rest away; Decompress scales them back up. The rounding error lands on the same noise budget as the lattice noise, and the parameters leave enough room that decryption still recovers the message. And the same pair at width 1 is how the 32-byte message gets into and out of a polynomial: Decompress₁ turns each of the 256 bits into a coefficient, 0 for a 0 bit, 1665 (that's q over 2, rounded up) for a 1 bit; Compress₁ goes back, rounding each coefficient to whichever of 0 and q over 2 it is closer to. Keep that picture; Encrypt and Decrypt lean on it.

SampleNTT and SamplePolyCBD, in `sampling.c`.

SampleNTT expands the public matrix A from the public seed, keeping only 12-bit chunks that are less than q so the coefficients stay uniform; its output counts as already in NTT form, which is why A is never transformed.

SamplePolyCBD builds the secret and noise polynomials: fixed-length pseudorandom function (PRF) output mapped through a centered binomial distribution (CBD): count some bits, subtract others, and you get small integers clustered around zero, from minus η to plus η . Small is the point: big enough to hide the secret, small enough that honest decryption still rounds back to the message.

SampleNTT may be variable-time; fine, because that seed is public. SamplePolyCBD must stay fixed-time because it samples secrets.

SLIDE 54

At last, the Number-Theoretic Transform (NTT). Remember Barrett replacing ordinary modulo at MLKEM_Q? Same idea at full scale, now Montgomery: a second divide-free reduction for the multiply-heavy inner loop.

Multiplying two 256-coefficient polynomials the ordinary way is slow. The NTT is a change of representation, an exact cousin of the Fourier transform, after which multiplication is nearly coefficient-by-coefficient. For this q it is incomplete: 3329 has roots of unity for seven halving layers, not eight, so it stops one layer short at 128 degree-one products. FIPS calls it essential: multiplication is defined in the NTT domain, and the encapsulation key ships in that form, so the transform is part of the wire format. Get it wrong and you don't interoperate.

And it's not just speed. This multiply runs on secret data, so it must not leak. The NTT is a fixed sequence of butterflies (same indices, same order, no secret-dependent branch or memory), and Montgomery reduction stays in a multiply and a shift, never a data-dependent divide.

One caveat: full constant-time also needs the chip's integer multiply to be constant-time, fine on mainstream cores, a gotcha on some small embedded ones, so the side-channel proofs attach to the backend you ship, not the math you read.

Not the whole story either: you'll also need constant-time comparisons and implicit rejection later. This is where it starts, and it's not an optimization to strip out; it's the math as standardized, and part of what keeps it from leaking.

Now the hash functions. SampleNTT already leaned on the extendable-output function (XOF); here's the object underneath. It's a sponge; the technical term, not a metaphor, and for all the plumbing, what it does is just hashing.

What is hashing for? A hash turns any input into fixed-length, random-looking bits, deterministically. So the scheme regenerates big values (matrix, noise, keys) from a tiny seed, and a whole public key collapses into one short binding tag.

What rules out SHA-2? Its design publishes the full internal state, so anyone can keep hashing onto the end; length extension, a trap for protocols, not a break of the hash. A sponge doesn't: part of that state, the capacity never leaves. So SHA-3 (KECCAK as FIPS 202) gets that for free, but the real reason ML-KEM builds on a sponge is that it keeps squeezing for as many bytes as you ask, a fixed-length digest can't. SHA-2 isn't weak; it's the wrong shape for this job.

Here's what matters for ML-KEM: one sponge, five hats.

Two are fixed-length SHA-3 digests: H is SHA3-256, G is SHA3-512.

Three keep squeezing: J and the PRF are SHAKE256, the XOF is SHAKE128.

One-line jobs, as on the card. H fingerprints a key. G stretches one input into two: at KeyGen, one seed into two seeds; at Encaps, the message into shared secret plus encryption randomness. J builds the decoy shared secret on a Decaps mismatch (implicit rejection). The PRF samples the secret CBD noise. The XOF expands matrix A, squeezing extra blocks whenever SampleNTT rejects, so A needs an extendable output and a fixed digest won't do.

The catch is cost: the sponge runs constantly, and without hardware-accelerated SHA-3 it can dominate the cycle count on constrained targets. That is what the config swap is for: a tuned, validated KECCAK.

Primitives done. Back to the top of the map, where the three layers above the auxiliaries light up: K-PKE, the internal algorithms, and the external ML-KEM you actually call.

The files that carry those layers: `mlkem_native.h`, `indcpa.c`, `kem.c`, plus `randombytes.h` and `verify.c`.

Alice and Bob, at last. One picture for the walk: a key-encapsulation mechanism, Figure 1 in FIPS 203. Three phases, two slides each: the diagram for what goes in, what comes out, and what makes it hold; then the code card, where the pieces get their letters, equations, and function names.

KeyGen is Alice's move. From fresh randomness she derives an encapsulation (public) key and a matching decapsulation (private) key. The decapsulation key stays with her. The encapsulation key goes to Bob in the clear, and the callout says what it is: packed noisy equations plus a 32-byte seed. Noisy equations with Alice's secret as the unknown, unsolvable through the noise: that is Module Learning with Errors. The seed lets both sides generate the same public coefficient matrix locally, so the matrix never goes on the wire. Now the code.

On the card: external into internal, then K-PKE. First the letters. The public coefficient matrix is A . Alice's secret is the vector s . The small errors are e . The published noisy equations are the vector t : t equals A times s plus e . Without e , linear algebra would solve for s ; with it, no known method does, classical or quantum. That is Module Learning with Errors (MLWE). Encapsulation key: t plus the seed for A . Secret in the decapsulation key: s . Hold those four letters; they carry the rest of the walk.

At the top, Algorithm 19, `mlk_kem_keypair`, is the KeyGen you ship. It pulls 64 fresh bytes through an approved random bit generator (RBG); in `mlkem-native`, that is your `randombytes`. Those bytes are seeds d and z . It hands them to Algorithm 16, then wipes the coins buffer.

Algorithm 16, `mlk_kem_keypair_derand`, is KeyGen with the coins in hand.

Job: turn d and z into the keypair.

Out: encapsulation key and decapsulation key.

How: calls `K-PKE.KeyGen`, then wraps it. The encapsulation key is the packed public vector plus the public seed. The decapsulation key holds the secret vector, a full copy of the encapsulation key so Decaps can re-encrypt, its 32-byte hash H (SHA3-256), and rejection

seed z . Copy, hash, and z all serve the Fujisaki-Okamoto (FO) wrapper, the re-encrypt-and-compare check we'll watch inside Decaps. z is copied into the decapsulation key here, so wiping the coins cannot erase it.

At the bottom, Algorithm 13, `K-PKE.KeyGen`, is the lattice core.

Job: from seed d , build the noisy Module Learning with Errors (MLWE) keypair.

Out: a K-PKE encryption key (packed t plus the seed for A) and a K-PKE decryption key (packed s).

How: G (SHA3-512) splits d , plus the rank k as one byte so two levels can never share a seed, into a public seed for A and a secret seed for the noise. The extendable-output function (XOF) (SHAKE128) expands A . The PRF (SHAKE256) samples the CBD for s and e . NTT s and e , form t as matrix times secret plus noise, Barrett-reduce, and pack s and t -plus-seed with lossless `ByteEncode_12`.

SLIDE 62

Encaps is Bob's move, and he never needs Alice's decapsulation key. From her encapsulation key he gets a ciphertext and his copy of the shared secret K . The callouts: he rebuilds the same public matrix A from the seed, transposed, as the equations need, completing Alice's noisy equations, then forms one more noisy equation with a fresh 32-byte message m hidden inside it. Compressed, that equation is the ciphertext, the only thing that crosses the wire.

Picture Romeo sealing a note in a box only Juliet can open. Except the box is a noisy linear equation, the note is 32 random bytes, and Juliet's secret is the one thing that cancels the hard part and reads the note through the leftover noise. K is hashed out of message m ; it never travels. Now the code.

SLIDE 63

Down the stack again. m is Bob's 32-byte message, r the randomness derived from it, y his ephemeral secret, e_1 and e_2 his errors, u and v the two halves of the ciphertext.

Algorithm 20, `mlk_kem_enc`, is the Encaps you ship. It pulls 32 fresh bytes through the same RBG: that is message m . Then it hands off to Algorithm 17, which first runs the modulus check on the encapsulation key: decode, re-encode, and the bytes must match, proving every coefficient is below q . FIPS files that check under Algorithm 20, Section 7.2; the library runs it one level down so both entry points get it.

Algorithm 17, `mlk_kem_enc_derand`, is Encaps with m in hand.

Job: encapsulation key plus m into ciphertext and shared secret.

Out: ciphertext and K .

How: hash the encapsulation key with H (SHA3-256); mix m and that hash through G (SHA3-512) into shared secret K and encryption randomness r ; call $K\text{-PKE.Encrypt}$; copy K out.

Algorithm 14, $K\text{-PKE.Encrypt}$, is the lattice core.

Job: encrypt m under the $K\text{-PKE}$ encryption key using r .

Out: the ciphertext bytes.

How, in three moves. First, inputs. Unpack the encryption key into Alice's t (already in NTT form) and the seed, and rebuild A -transpose from that seed with the extendable-output function (XOF), exactly as Alice built A . Encode m into a polynomial μ with the width-1 decompress: 256 bits, one per coefficient; a 0 bit stays 0, a 1 bit becomes 1665, about q over 2, as far apart as the modulus allows so small noise cannot flip one into the other as we already covered twice. Sample y , e_1 , e_2 with the PRF, seeded from r . That seeding is what makes this encryption reproducible, which Decaps will need.

Second, the lattice math, as NTT multiply-accumulates and inverse NTTs. u equals A -transpose times y plus e_1 : k polynomials, Bob's new noisy equation with y as its unknown. v equals t times y plus e_2 plus μ : one polynomial, the right-hand side with the encoded message on top. Since t is A times s plus noise, v is very nearly s times u plus μ plus small noise. Only someone holding s can compute s times u and subtract it away.

Third, output. Barrett-reduce, lossy-compress u with du bits and v with dv bits, pack. That is the ciphertext: 1,088 bytes at ML-KEM-768. Not in it: K . That came out of $G(m \parallel H(ek))$ in Algorithm 17, and Bob keeps it. The ciphertext carries only m , hidden inside v .

SLIDE 64

Decaps is Alice's move, and it is deterministic: no fresh randomness, no randombytes call. From her decapsulation key and Bob's ciphertext she gets her copy, K' . The callout's three lines are what she does: subtract with her secret to recover Bob's 32-byte message m from that extra noisy equation; derive K' with the same hash Bob used, so on honest runs it matches his K with overwhelming probability, FIPS's phrase for almost never fails; and re-encrypt to check the ciphertext is genuine, answering a mismatch with a decoy, not an error. Still, a finished decaps is not proof the ciphertext was the one Bob sent: the protocol owns integrity; the KEM owns key agreement. Now the code.

Last of the three, and where Fujisaki-Okamoto shows up in the path. One new letter, z , the rejection seed KeyGen tucked into the decapsulation key, and one new hash, J .

Algorithm 21, `mlk_kem_dec`, is the Decaps you ship. No derand twin: Decaps is deterministic, so Algorithm 18's internal decaps folds straight into this one function.

Job: decapsulation key plus ciphertext into Alice's shared secret.

Out: K' .

How: first the hash check on the decapsulation key (Section 7.3): re-hash the encapsulation key stored inside it and compare to the stored H , catching a corrupted key before any secret math runs. Then decrypt to a candidate m' . The FO path (re-derive, re-encrypt, compare, implicit reject) is below.

Algorithm 15, `K-PKE.Decrypt`, is the lattice core.

Job: open the ciphertext with the K-PKE decryption key.

Out: a candidate m' .

How: decompress the ciphertext back into u and v , unpack s . Compute the one thing only Alice can, s times u , as an NTT multiply-accumulate plus inverse NTT. Subtract it from v and the lattice part cancels, leaving mu plus a small pile of noise: Alice's e , Bob's e_1 and e_2 , and the compression rounding. Barrett-reduce, then round each coefficient to the nearer of 0 and q over 2. Those 256 bits are the candidate m' . Not yet the shared secret, and not yet trusted.

Then the FO bridge. Re-derive candidate K and randomness r with G , the same $G(m' || H(ek))$ mix Encaps used. Re-encrypt. Compare in constant time, no early exit. Then, match or not, always build the rejection secret with J (SHAKE256 of z and the ciphertext) and pick real or reject with a constant-time move, so a mismatch changes the answer but never the work done. That is implicit rejection. A wrong ciphertext does not trip an error; it opens onto a decoy room of 32 plausible bytes. A return of zero means the routine finished, not that the ciphertext was Bob's. Your logs must not become the oracle: anything that tells the attacker whether a guess landed.

Outlook: this is bare key agreement, and a real deployment wraps more around it. SP 800-227 names the scaffolding; guard keys and seeds, never leak Decaps failures, key confirmation, proof of possession for static keys, and hybrid combiners that bind both secrets plus their ciphertexts and encapsulation keys rather than just hashing the two halves together.

The same guidance owns what you do with the 32-byte shared secret itself, usually via an approved key-derivation function (KDF). Not today's code, but know it exists, and analyze the one you ship.

Step back: what makes that a KEM rather than just encryption is the Fujisaki-Okamoto (FO) transform: the re-encrypt-and-compare in Decaps, with implicit rejection on a mismatch.

Here's the stack. Module Learning with Errors (MLWE) makes the Kyber public-key encryption core (K-PKE) safe against a passive eavesdropper. That core is not safe against an attacker who sends ciphertexts to Decaps and learns from each response: an adaptive chosen-ciphertext attack, the decryption-oracle kind where each error response leaks a little more about the plaintext, and the oracle is at the protocol layer, no power probes needed; the CCA we flagged on the q slide.

So FO is the bridge: because the ciphertext is a deterministic function of m and the recipient's key, Decaps re-derives it and rejects anything that doesn't match, so probing queries return only useless bytes.

FO also leans on that lattice hardness and the near-zero failure rate from the MLKEM_Q slide, but it's what carries K-PKE from Indistinguishability under Chosen-Plaintext Attack (IND-CPA) up to the Indistinguishability under Adaptive Chosen-Ciphertext Attack (IND-CCA2) KEM the standard approves. One more callout: treat that inner core (indcpa.c) as a building block, never a product API; on its own it is not ML-KEM.

We're zooming out again. The line-by-line walk is behind us; from here it's recaps and summaries, pulling the threads together as the talk nears its end.

One card for where the C goes beyond the pseudocode: security on one side, speed on the other.

The NTT is the standout: a fixed, branch-free shape that leaks no timing, the change of representation that makes multiplication fast, and the on-the-wire format all at once; strip it and you either leak or you don't interoperate.

Barrett and Montgomery reduce modulo q with no data-dependent divide, in fixed time.

Sampling splits on who owns the data: SampleNTT may reject in variable time because the seed is public, while the noise sampler runs fixed-time because it touches the secret. Expanded A is public from the encapsulation key, so FIPS Sec 3.3 allows caching it; mlkem-native does not: it re-expands from the seed on every call and spends its cycles speeding that expansion instead, via 4x batched Keccak/SHAKE.

Compression buys the small ciphertext; the constant-time compare with implicit rejection closes the decapsulation oracle. Read it once and the pattern sticks: most of these earn their place twice, for safety and for speed.

Still zooming out: the requirements layer, the part of FIPS 203 with no algorithm numbers, Section 3.3, plus the Section 2 housekeeping for how the data is represented. We've already met most of it in pieces. Now we pin that checklist in one place: the rules every ML-KEM implementation has to meet before you even call KeyGen.

Here it is on the card: three columns. What FIPS fixes, what it leaves to you, and the shalls on top.

Mandates: representation and how to read the pseudocode; the data model (integers modulo q as Module Learning with Errors (MLWE) vectors and matrices); the arithmetic (add, NTT multiply, integer-only); the lifecycle (wipe intermediates; mlkem-natives zeroizes KeyGen coins). Keep these seeds only if you guard them like a decapsulation (private) key. Caching expanded A is fine; it is public from the encapsulation (public) key.

Flexibility: any steps that match on every input, randomness included, conform. Residue layout, reduction, assembly, batched hashing, Barrett or Montgomery: yours. Same bytes out, still ML-KEM. That is the blessing for the backends we toured.

Shalls on top, plus one strong should: keep K-PKE private; ship the randomized wrappers and keep the derand entry points for testing (FIPS words that one as a should; the shall is that the module itself does the random sampling); use an approved Random Bit Generator (RBG) at the right strength; run the input checks (check_pk, check_sk in the library); treat the shared secret as key material, not a session key. Use it directly only under the right conditions, or derive via SP 800-108 / 800-56C, and analyze any hybrid combiner on its own.

Meet every line and you have conforming ML-KEM. Whether it is safe in your product is still on you.

SLIDE 71

So, close the loop on the title.

What makes post-quantum crypto so hard to ship?

Not the lattice math. It's the engineering around a newer standard: kilobyte keys where we had 32 bytes, a hybrid handshake that trips middleboxes, constrained devices that were never sized for it, and the quiet truth that a flawless algorithm can still be bypassed if the handling around it leaks. That's all ops, and ops is exactly where reading the library pays off.

How do you read and judge open-source ML-KEM?

Spec-first, but not spec-only. Map each FIPS section to a file and each algorithm number to a function, and know which rewrites are load-bearing (the NTT, the constant-time compare, implicit rejection) versus which are free to swap. Then read everything around the algorithm: the config flags, the glue code, the CI and known-answer tests, and the performance knobs we toured (backends, stack budgets, the KECCAK cost). That scaffolding tells you as much about how ML-KEM behaves in production as the core math does.

Three things to carry out the door.

One: ML-KEM is a KEM: 32 bytes of shared secret you feed a protocol, wrapped by Fujisaki-Okamoto so it stays secure even against an attacker who feeds it forged or tampered ciphertexts.

Two: the code's shape is the security: constant-time, integer-only, implicit rejection are the standard, not optimizations.

Three: conformance is the floor; your RNG, your zeroize, your key handling decide whether it's actually safe.

That's a wrap. The whole thing (slides and this voice-over transcript) is in the GitHub materials repo (<https://github.com/neroduality/materials>). Open the ML-KEM code when you're ready, and know what to look for.